

7-4 · MinIO·Qdrant 연동 · 실습 1H

MinIO 버킷 구성과 문서 적재 실습

MinIO 를 띄우고 파이썬으로 버킷·객체를 다룬다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① `docker run` 으로 MinIO 를 띄우고 웹 콘솔(9001)에 로그인한다
- ② `raw-docs` 버킷을 만들고 7-2 의 notes 문서 두 개를 올린다
- ③ 일부러 `secure=True` 로 붙어 SSL 오류를 직접 본다
- ④ `get_object` 로 읽고 `close().release_conn()` 을 `finally` 에 둔다

4·6주차에서 띄운 MinIO 가 아직 떠 있으면 `port is already allocated` — 먼저 내려 주세요

여섯 단계 — 컨테이너 하나, 버킷 하나, 파이썬 객체 mc 하나

7-2 에서 만든 notes.txt·long_notes.txt 가 있는 폴더에서 시작합니다

1. MinIO 기동

docker run
pip install minio

2. 버킷·업로드

raw-docs
notes/ 두 개

3. 일부러 실패

secure=True

4. 읽기

get_object
release_conn()

5·6. 없는 키·prefix

NoSuchKey
접두사로 훑기

docker run 한 줄 — 옵션마다 뜻

옵션	뜻
<code>docker run -d --name minio</code>	백그라운드로, 이름은 minio
<code>-p 9000:9000 -p 9001:9001</code>	9000 = API · 9001 = 웹 콘솔
<code>-e MINIO_ROOT_USER=minioadmin</code>	로그인 계정 = 코드의 access_key
<code>-e MINIO_ROOT_PASSWORD=minioadmin</code>	비밀번호 = 코드의 secret_key
<code>-v minio_data:/data</code>	데이터를 담는 볼륨
<code>pgsty/minio server /data --console-address ":9001"</code>	서버 실행 · 콘솔 주소 지정

확인 포인트: `http://localhost:9001` 에 `minioadmin / minioadmin` 으로 로그인되면 성공

`docker ps --filter name=minio` 로 확인 · `pip install minio` · 4주차에 띄웠던 것이면 `docker start minio` 로 다시 켜다

버킷을 만들고 문서를 올린다 — 확인은 콘솔에서도

1 `mc = Minio("localhost:9000", ..., secure=False)`
로컬 컨테이너는 HTTP 이므로 `secure=False` — `access_key`·`secret_key` 는 `minioadmin`

2 `bucket_exists` → 없으면 `make_bucket("raw-docs")`
버킷은 한 번만 만든다 — 두 번째 실행에도 오류가 없게

3 `fput_object(BUCKET, "notes/notes.txt", "notes.txt")`
7-2 의 `notes.txt`·`long_notes.txt` 두 개를 `notes/` 아래로 올린다

4 `list_objects(BUCKET, recursive=True)`
두 객체와 크기(bytes)가 찍힌다 — 웹 콘솔 `raw-docs` 에서도 같은 객체

확인 포인트

객체 두 개가
콘솔에도

secure=True 로 붙어 본다 — 한 줄이 몇 시간을 먹는다

secure=False — 로컬 컨테이너

로컬 MinIO 는 HTTP — bucket_exists 가 답을 준다

- Minio("localhost:9000", ..., secure=False)
- 이번 실습의 기본값
- 4주차의 MinIO 코드와 같다

VS

secure=True — 운영 HTTPS 용

로컬에 붙이면 SSL 관련 오류가 난다

- bad = Minio(..., secure=True)
- bad.bucket_exists(BUCKET) → 예외
- except 에서 type(e).__name__ 과 str(e)[:200] 출력

읽고, 반드시 돌려주고, 없는 키를 읽어 본다

객체 키는
영문으로
통일

1 `body = mc.get_object(BUCKET, "notes/notes.txt")`
body.read().decode("utf-8") 로 원문 — len 과 앞 100자를 찍는다

2 `finally: body.close(); body.release_conn()`
오류가 나도 연결을 반드시 돌려준다 — 반복문에서 빼면 곧 연결 고갈

3 `get_object(BUCKET, "notes/없는파일.txt")`
NoSuchKey 계열 오류 — type(e).__name__ 과 e.code 를 출력해 본다

4 `keys = [o.object_name for o in list_objects(...)]`
실제 키를 먼저 찍어 보는 습관 — 한글 키는 NFC/NFD 차이로 어긋난다

접두사(prefix)로 범위를 좁힌다 — 4주차 raw/orders/ 처럼

전체 — recursive=True

manual/guide.txt 를 하나 더 올리면 세 객체

- notes/notes.txt
- notes/long_notes.txt
- manual/guide.txt

VS

prefix="notes/" — 범위 좁히기

notes/ 아래 두 객체만 훑는다

- notes/notes.txt
- notes/long_notes.txt
- 종류·부서·연도별 접두사 → 「이 폴더만 재색인」

이 1시간이 끝날 때 확인할 것

- 1 localhost:9001 에 minioadmin / minioadmin 으로 로그인된다
- 2 list_objects 에 두 객체와 크기가 찍히고 콘솔에서도 보인다
- 3 secure=True 로 붙으면 SSL 관련 오류가 난다
- 4 get_object 의 원문이 찍히고 close().release_conn() 이 finally 에 있다
- 5 없는 키는 NoSuchKey · prefix="notes/" 는 두 객체만 나온다

7-4 · 실습 1H 마무리

다음은 MinIO → 임베딩 → Qdrant 파이프라인 (1.5H)

버킷을 훑어 자르고 임베딩해 Qdrant 에 넣는 ingest() 를 만들고, 재실행해도 늘지 않는지 assert 로 잡니다

✓ ingest() — 순회 → 자르기 → 임베딩 → upsert

✓ 재실행 전후 count 를 assert 로 비교

✓ uuid4 로 바꿔 중복이 쌓이는 것을 재현

✓ source 로 원본 역참조 · 줄어든 문서 재적재

가져갈 것

mc 객체 · raw-docs 버킷 · 7-3 의 emb